Eingebaute Bilderkennung für Wildkameras

Fine-Tuning von YOLO26l und Edge-KI-Inferenz auf dem Raspberry Pi 5

Rickardo Dimbison Doré

Aufgabenstellung

Ein auf einem Raspberry Pi 5 basierendes Wildkamerasystem **mit lokal ausgeführter Bilderkennung auf Basis von YOLO26.**

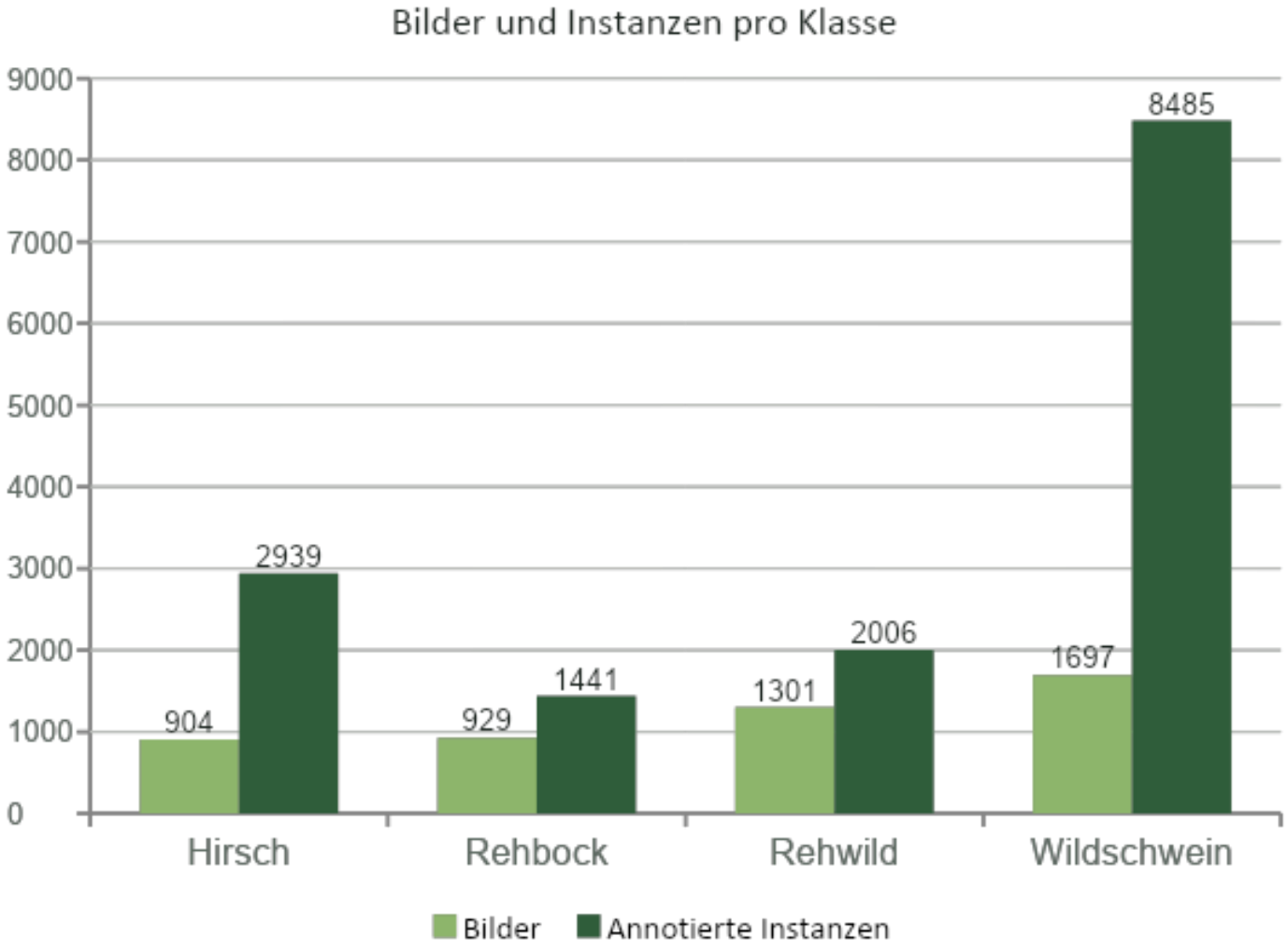


Offline-First: läuft vollständig ohne Netzverbindung

optionale Ergänzung

Umfang der Arbeit: eigener Wildtier-Datensatz und Fine-Tuning von YOLO26l · Export nach NCNN und Inferenz auf der Pi-5-CPU · Prototyp aus Kamera, PIR-Sensor und 4G-Modul · Software mit lokaler Warteschlange, SMS-Benachrichtigung und Fernsteuerung

Eigener Wildtier-Datensatz



4.831

Bilder von SPYPOINT-Kameras

4

Tierklassen

57,1 %

Instanzen: Wildschwein

Klassenungleichgewicht: Rehbock nur 1.441 Instanzen – später sichtbar in den Ergebnissen.

Trainingskonfiguration

220
Epochen

H100
80 GB · Google Colab

AdamW
Optimierer

0,001
Lernrate lr0 (Cosine)

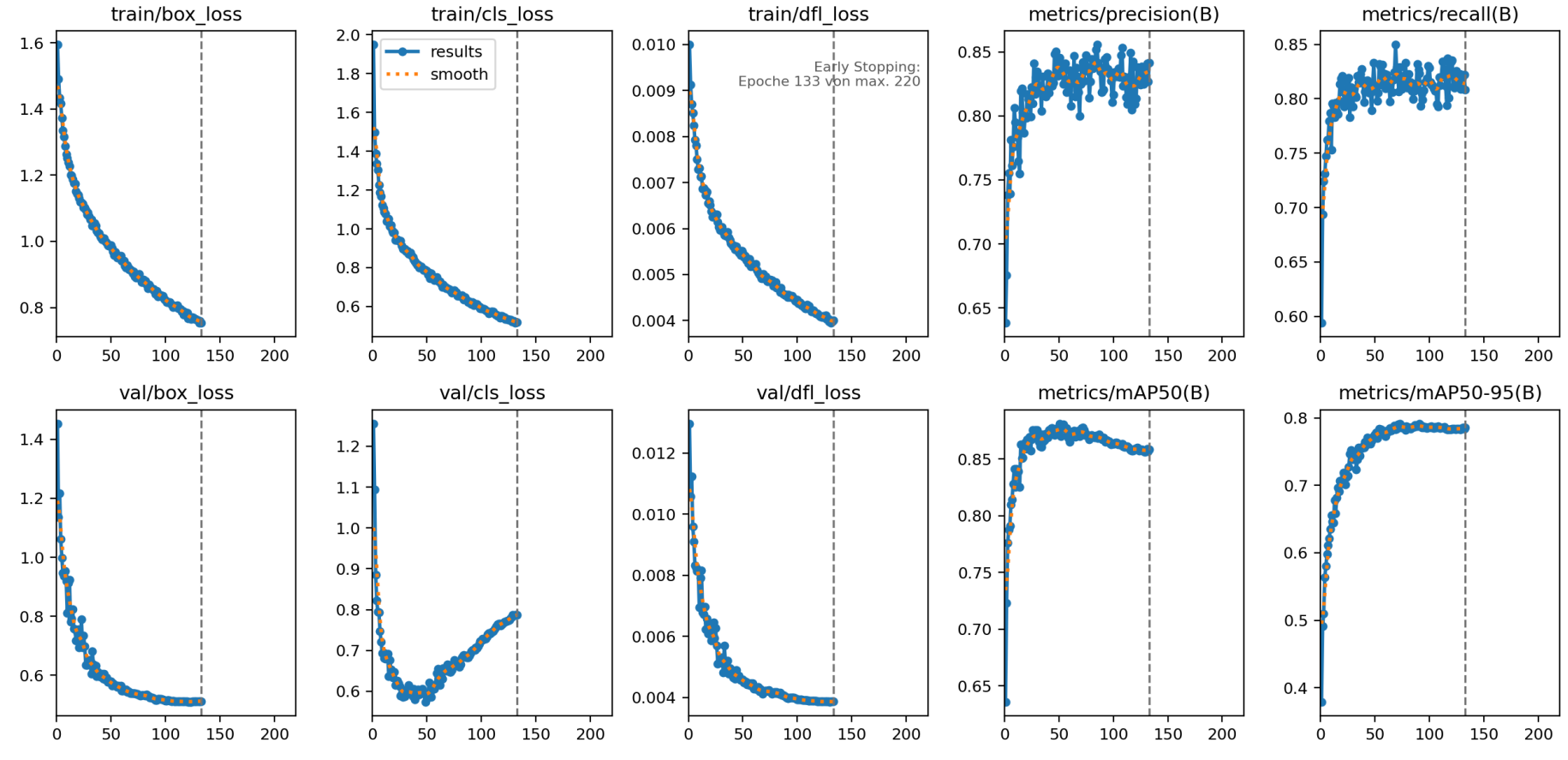
Optimierung & Regularisierung

- Niedrige Lernrate schützt vor catastrophic forgetting
- 5 Warmup-Epochen, Cosine-Annealing bis 0,0001
- Weight Decay 0,01 und Early Stopping (patience = 60)
- Startpunkt: yolo26l.pt (COCO-Gewichte)

Augmentierung im Training

- Mosaic 100 % (in den letzten 15 Epochen aus)
- MixUp 12 % · Copy-Paste 15 %
- Geometrie: Rotation ±8°, Skalierung ±75 %, Spiegelung 50 %
- HSV-Helligkeit ±40 % simuliert schwankende IR-Beleuchtung

Trainingsverlauf: stabile Konvergenz



Oben Trainingsdaten, unten Validierungsdaten · Early Stopping nach 133 von maximal 220 Epochen

Stabile Konvergenz

Alle Verlustkurven sinken gleichmäßig – die Lernrate war passend gewählt.

Plateau erreicht

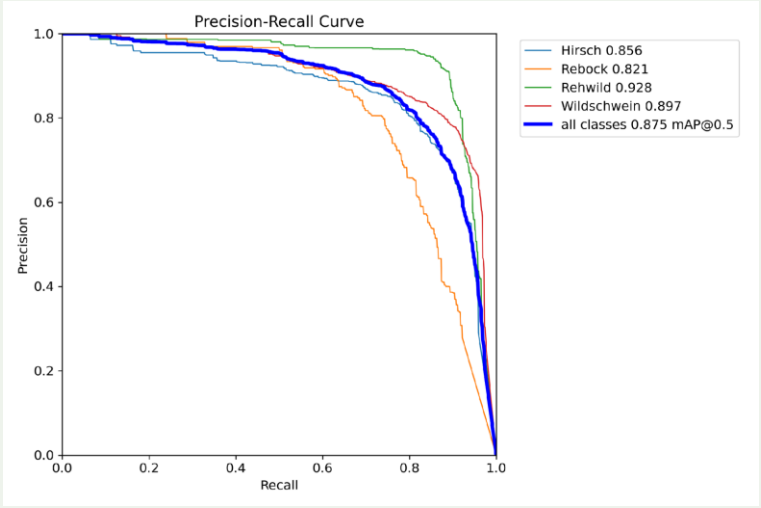
Die Kennzahlen laufen früh in ein Plateau: Die Grenze liegt bei den Daten.

Overfitting bei der Klassifikation

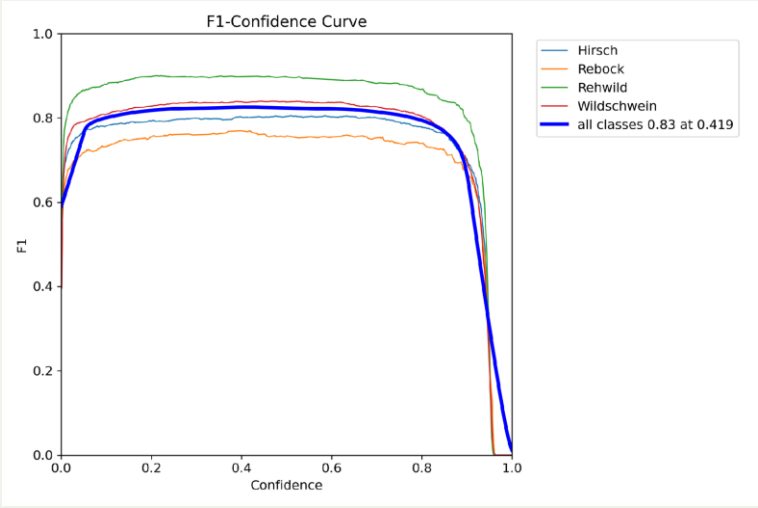
val/cls_loss steigt ab Epoche 50 wieder an – gespeichert wird das beste Modell.

Bewertung auf Klassenebene

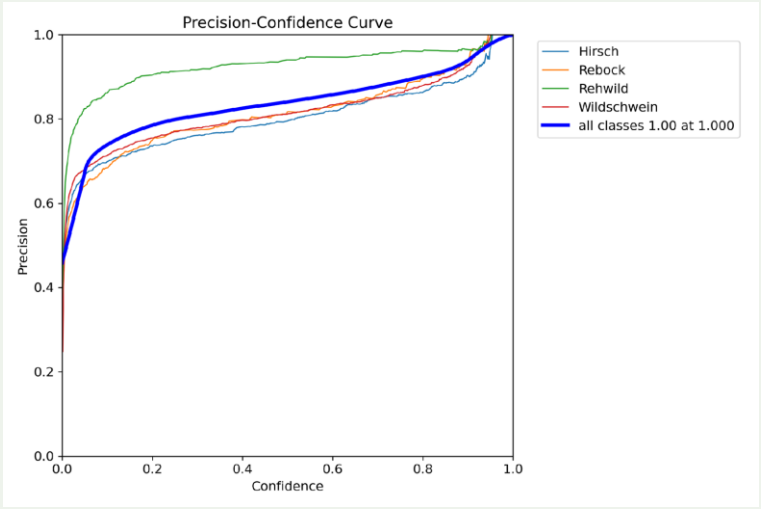
Precision-Recall



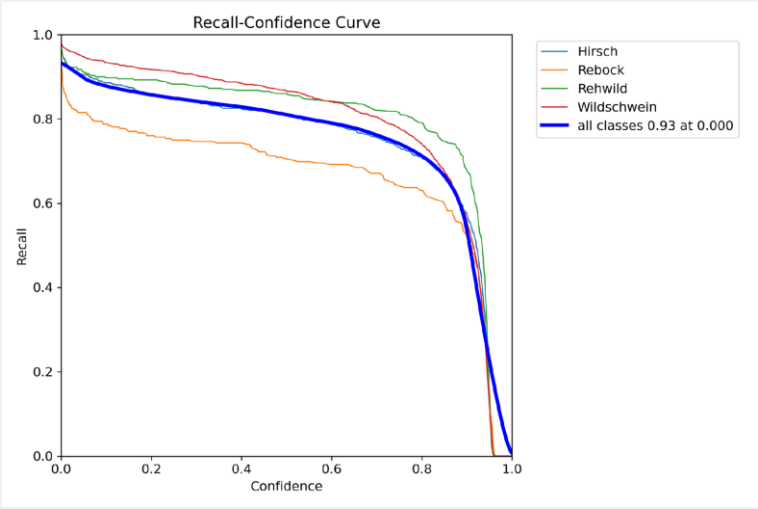
F1-Confidence



Precision-Confidence



Recall-Confidence



Was die Kurven zeigen

Precision-Recall

Die Fläche unter der Kurve ist die AP je Klasse: Rehwild 0,93 oben, Rebock 0,82 unten – mAP@0.5 = 0,875

F1-Confidence

Bestes Gleichgewicht bei conf = 0,419 mit F1 = 0,83; das Plateau ist breit

Precision-Confidence

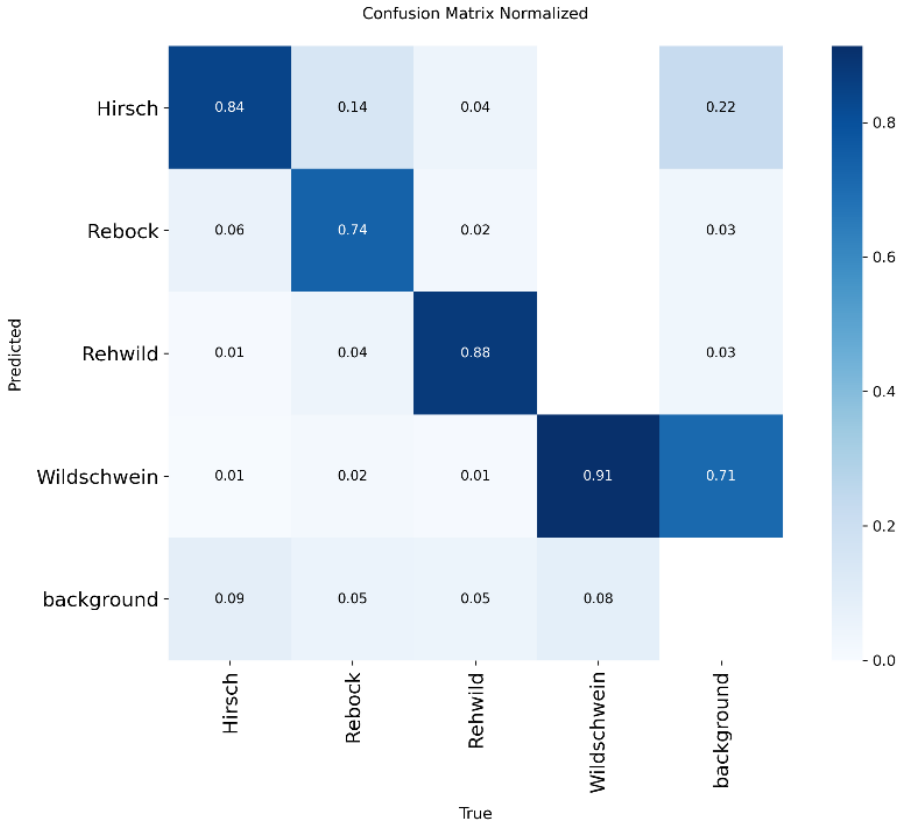
Höhere Schwelle → weniger Fehlalarme

Recall-Confidence

Höhere Schwelle → mehr übersehene Tiere

Betriebspunkt 0,4–0,5
Kompromiss aus Fehlalarm und Erkennung

Konfusionsmatrix: wo das Modell irrt



Normalisierte Konfusionsmatrix (spaltenweise)

Recall pro Klasse

Wildschwein 91 % · Rehwild 88 % · Hirsch 84 % · Rehbock 74 %

Verwechslung Hirsch ↔ Rehbock

14 % der Hirsche als Rehbock, 6 % umgekehrt – ähnliche Silhouette nachts

Bias zu Wildschwein

71 % der Hintergrund-Fehlalarme → 396 FP-Detektionen als Wildschwein

Ursache liegt in den Daten, nicht in der Architektur → Oversampling empfohlen

Systemarchitektur und Datenfluss

Edge	Pi 5 · Kamera · PIR · YOLO-NCNN · lokale Speicherung
Kommunikation	SIM7600: GSM-Kanal (SMS) + 4G-Datenkanal
Cloud (optional)	Azure IoT Hub (Direct Methods) · Blob Storage

Datenfluss pro Ereignis

Erfassung → **Inferenz 640², conf 0,5** → **Entscheidung** → **Persistenz DCIM/ + TRANSMIT/** → **Upload Blob Storage**

Kein Tier erkannt → Aufnahme wird verworfen. Bilder ohne Tierbezug verlassen das Gerät nie (Datenminimierung auf Architekturebene).

Software: modular, nebenläufig, offline-first

4 Thread-Typen (Python 3)



PIR-Thread

wartet ereignisbasiert, Cooldown, startet Aufnahme



Upload-Queue-Thread

prüft alle 60 s die 4G-Verbindung, lädt chronologisch hoch



IoT-Hub-Thread

Reconnect mit exponentiellem Backoff 5–300 s



SMS-Threads

kurzlebig – blockieren die Kamera nicht



Kamera-Mutex

PIR blockierend, Fernbefehl nicht-blockierend → HTTP 503, wenn belegt



Offline-First

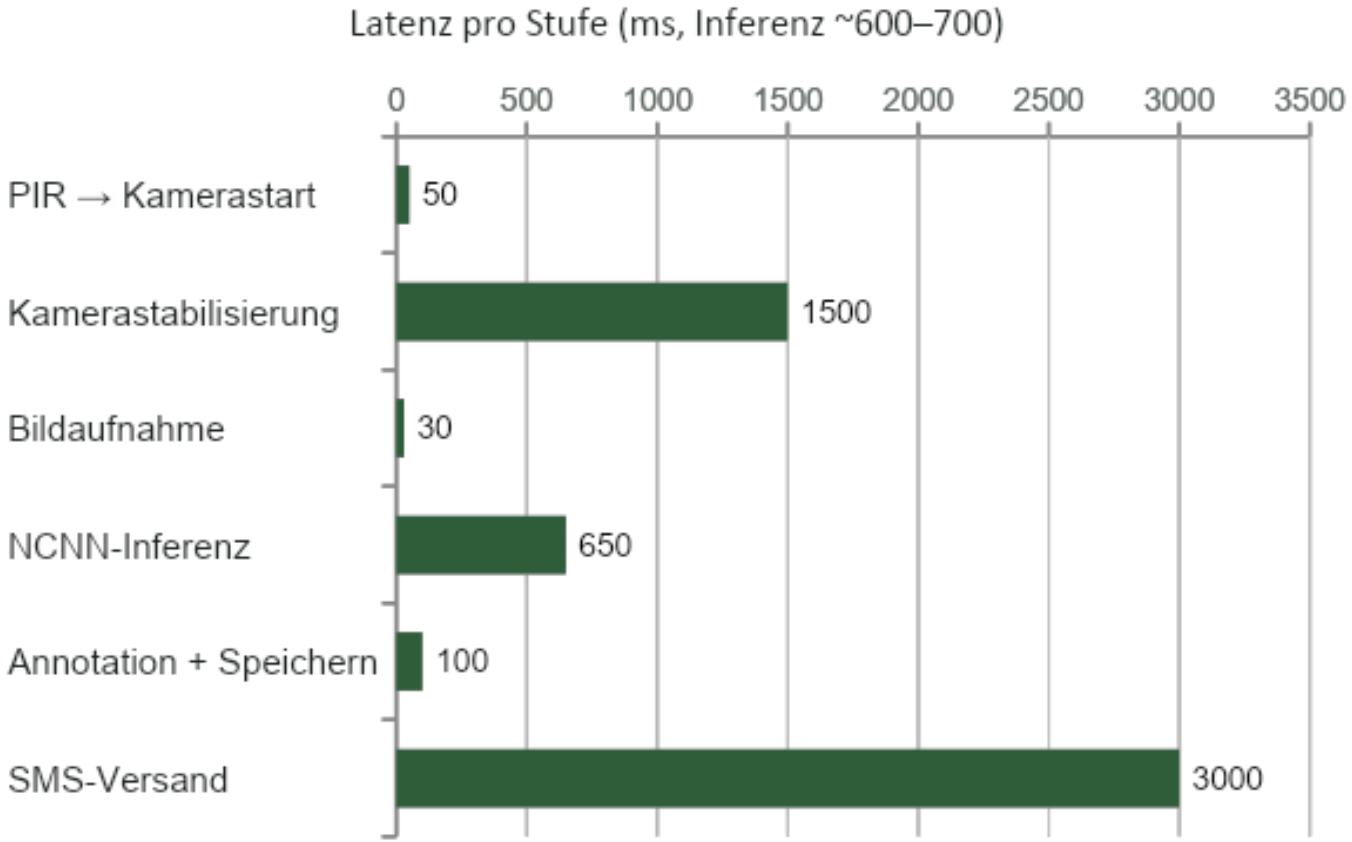
Erkennung, Speicherung und SMS laufen ohne Cloud; Warteschlange überlebt Neustart



Sicherheit

Secrets nur in .env (nicht versioniert) · Eingabevalidierung (400/404/503)

Systemtest: Latenz und Robustheit



≈ 5,3–6,1 s PIR → SMS

Offline-/Reconnect-Tests

Szenario 1

Aufnahmen ohne 4G → lokal gepuffert, SMS kommt an

Szenario 2

Verbindung zurück → Upload chronologisch, keine Verluste/Duplikate

Szenario 3

IoT-Hub-Ausfall → Backoff, lokaler Betrieb unbeeinträchtigt

Irrelevante Aufnahmen werden vor der Übertragung verworfen – weniger Datenvolumen und Kosten.

YOLO26l läuft vollständig lokal auf der Pi-5-CPU; Inferenz ist latenzseitig unkritisch.

mAP@0.5 = 0,875 auf den Wildtierdaten

Mehrwert des Ansatzes

- vier Arten statt nur „Tier erkannt“
- Bilder ohne Tier verlassen das Gerät nie
- Erkennung und SMS auch ohne Datenverbindung